Universidad

Sistema de Reserva de Espacios Universitarios

Informe de Formulación y Diseño de Software

Asignatura:	Ingeniería de Software
Estudiante:	Nombre del estudiante
Docente:	Nombre del docente
Carrera:	Ingeniería Civil Informática
Fecha:	13 de octubre de 2026

1. Resumen Ejecutivo

El presente informe desarrolla la formulación y diseño de un Sistema de Reserva de Espacios Universitarios, cuyo propósito es facilitar la consulta de disponibilidad, solicitud, confirmación y gestión de espacios institucionales.

La solución considera distintos perfiles de usuario, incluyendo estudiantes, docentes y administradores. El sistema centraliza información relacionada con usuarios, espacios, horarios, cursos, reservas y notificaciones.

El proyecto comprende el levantamiento de requerimientos funcionales y no funcionales, modelamiento mediante UML, diseño conceptual, arquitectónico y detallado, construcción de un prototipo TRL-2 y definición de una estrategia de pruebas.

Como resultado se propone una arquitectura modular que separa interfaz, API, lógica de negocio y persistencia, permitiendo evolucionar posteriormente hacia una implementación funcional.

Índice

1. Resumen Ejecutivo **1**

2. Introducción **4**

2.1. Objetivo General 4

2.2. Objetivos Específicos 4

2.3. Alcance 4

3. Descripción del Problema **4**

4. Requerimientos **5**

4.1. Requerimientos Funcionales 5

4.2. Requerimientos No Funcionales 6

5. Diagrama de Casos de Uso **6**

6. Diagrama de Flujo de Procesos **7**

6.1. Diagrama de Clases 8

6.2. Modelo Entidad–Relación 9

7. Modelo de Descripción Detallada de los Requerimientos **9**

7.1. RF07 – Crear Reserva 10

8. Diseño Conceptual del Software **10**

8.1. Casos de Uso Extendidos 10

8.1.1. CU07 – Crear Reserva 11

8.2. Diagrama de Datos Relacional 11

9. Diseño Arquitectónico **12**

9.1. Arquitectura del Sistema 12

9.2. Diagrama de Componentes 13

10. Diseño Detallado **13**

10.1. Diagrama de Secuencia 13
10.2. Diagrama de Estado 14
10.3. Diagrama de Actividad 14

11.Prototipo TRL-2 **15**

12.Diseño de Pruebas **16**
12.1. Estrategias de Pruebas 16
12.1.1. Modelo de Caso de Prueba 16
12.2. Resultados de Pruebas 16

Matriz de Trazabilidad **17**

13.Conclusiones **17**

14.Referencias **18**

2. Introducción

La gestión de espacios universitarios constituye un proceso que involucra estudiantes, docentes y unidades administrativas. Salas, laboratorios y otros recursos deben asignarse considerando disponibilidad, horarios y actividades académicas.

Cuando esta información se encuentra distribuida o depende de procedimientos manuales, pueden producirse dificultades para conocer la disponibilidad real de los espacios, conflictos entre reservas y mayores tiempos de coordinación.

El proyecto propone el diseño de un Sistema de Reserva de Espacios Universitarios que centralice estas operaciones mediante una plataforma digital.

2.1. Objetivo General

Diseñar un sistema de software que permita gestionar de manera centralizada la disponibilidad y reserva de espacios universitarios.

2.2. Objetivos Específicos

1. Identificar los requerimientos funcionales y no funcionales.
2. Modelar los procesos mediante diagramas UML.
3. Diseñar el modelo de datos de la solución.
4. Definir una arquitectura modular para el sistema.
5. Elaborar un prototipo TRL-2.
6. Diseñar una estrategia de pruebas para validar los requerimientos definidos.

2.3. Alcance

El sistema considera gestión de usuarios, autenticación, consulta de disponibilidad, reservas, cancelaciones, administración de espacios y horarios, reservas asociadas a cursos, notificaciones y generación de reportes.

3. Descripción del Problema

La administración de espacios universitarios requiere coordinar usuarios, recursos físicos, fechas y horarios.

La ausencia de una plataforma centralizada puede dificultar:

- conocer la disponibilidad actual de los espacios;
- evitar conflictos entre reservas;
- mantener un historial de solicitudes;
- administrar horarios;
- comunicar confirmaciones;
- relacionar reservas con actividades académicas;
- generar información para la gestión.

Por esta razón, se plantea desarrollar una solución que permita digitalizar y centralizar el proceso.

4. Requerimientos

Se identificaron **15 requerimientos funcionales** y **10 requerimientos no funcionales**.

4.1. Requerimientos Funcionales

ID	Requerimiento
RF01	El sistema deberá permitir registrar usuarios.
RF02	El sistema deberá autenticar usuarios mediante inicio de sesión.
RF03	El administrador deberá poder gestionar usuarios.
RF04	El sistema deberá gestionar roles y permisos.
RF05	El usuario deberá poder consultar espacios disponibles.
RF06	El sistema deberá verificar la disponibilidad de un espacio para una fecha y horario determinados.
RF07	El usuario deberá poder crear una reserva.
RF08	El sistema deberá confirmar una reserva realizada.
RF09	El usuario deberá poder cancelar una reserva.
RF10	El usuario deberá poder consultar sus reservas.
RF11	El administrador deberá poder gestionar espacios universitarios.
RF12	El administrador deberá poder configurar horarios.

ID	Requerimiento
RF13	El sistema deberá permitir gestionar reservas asociadas a cursos.
RF14	El sistema deberá enviar notificaciones de confirmación.
RF15	El administrador deberá poder generar reportes de reservas y utilización de espacios.

4.2. Requerimientos No Funcionales

ID	Categoría	Requerimiento
RNF01	Rendimiento	Las consultas de disponibilidad deberán responder en un máximo de 2 segundos bajo condiciones normales.
RNF02	Disponibilidad	El sistema deberá alcanzar una disponibilidad mensual mínima del 99 %, excluyendo mantenimientos programados.
RNF03	Seguridad	Las contraseñas deberán almacenarse utilizando mecanismos seguros de hash.
RNF04	Seguridad	Las funcionalidades deberán restringirse según los roles y permisos asignados.
RNF05	Integridad	El sistema deberá impedir reservas incompatibles para un mismo espacio, fecha y horario.
RNF06	Usabilidad	La interfaz deberá presentar mensajes comprensibles de confirmación, error y alternativas.
RNF07	Compatibilidad	La aplicación web deberá funcionar en navegadores web modernos.
RNF08	Persistencia	Las reservas confirmadas deberán almacenarse correctamente en PostgreSQL.
RNF09	Mantenibilidad	La solución deberá mantener separación entre interfaz, API, servicios, repositorios y persistencia.
RNF10	Confiabilidad	Una reserva no deberá informarse como confirmada cuando la operación de persistencia no haya finalizado correctamente.

5. Diagrama de Casos de Uso

Los principales actores identificados son:

- Estudiante.
- Docente.
- Administrador.
- Sistema de Notificaciones.

El modelo representa las principales funcionalidades y las relaciones de inclusión y extensión existentes entre ellas.

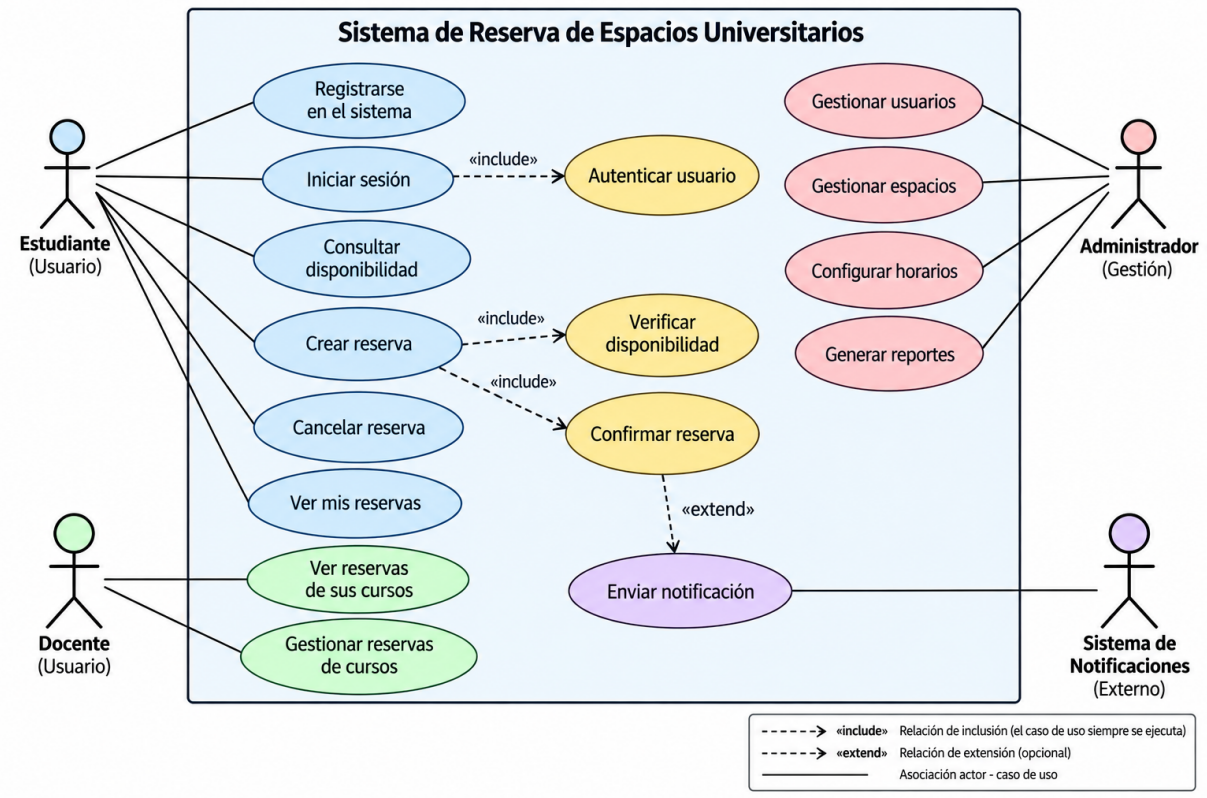


Figura 1: Diagrama de casos de uso del sistema.

6. Diagrama de Flujo de Procesos

El proceso principal corresponde a la creación de una reserva.

De manera general:

$$Usuario \rightarrow Consulta \rightarrow Disponibilidad \rightarrow Solicitud \rightarrow Validacin \rightarrow Reserva \rightarrow Confirmacin$$

6.1. Diagrama de Clases

El diagrama de clases representa las principales entidades del dominio y sus responsabilidades.

Las clases principales son:

- Usuario.
- Rol.
- Reserva.
- Espacio.
- Horario.
- Curso.
- ReservaCurso.
- Notificación.

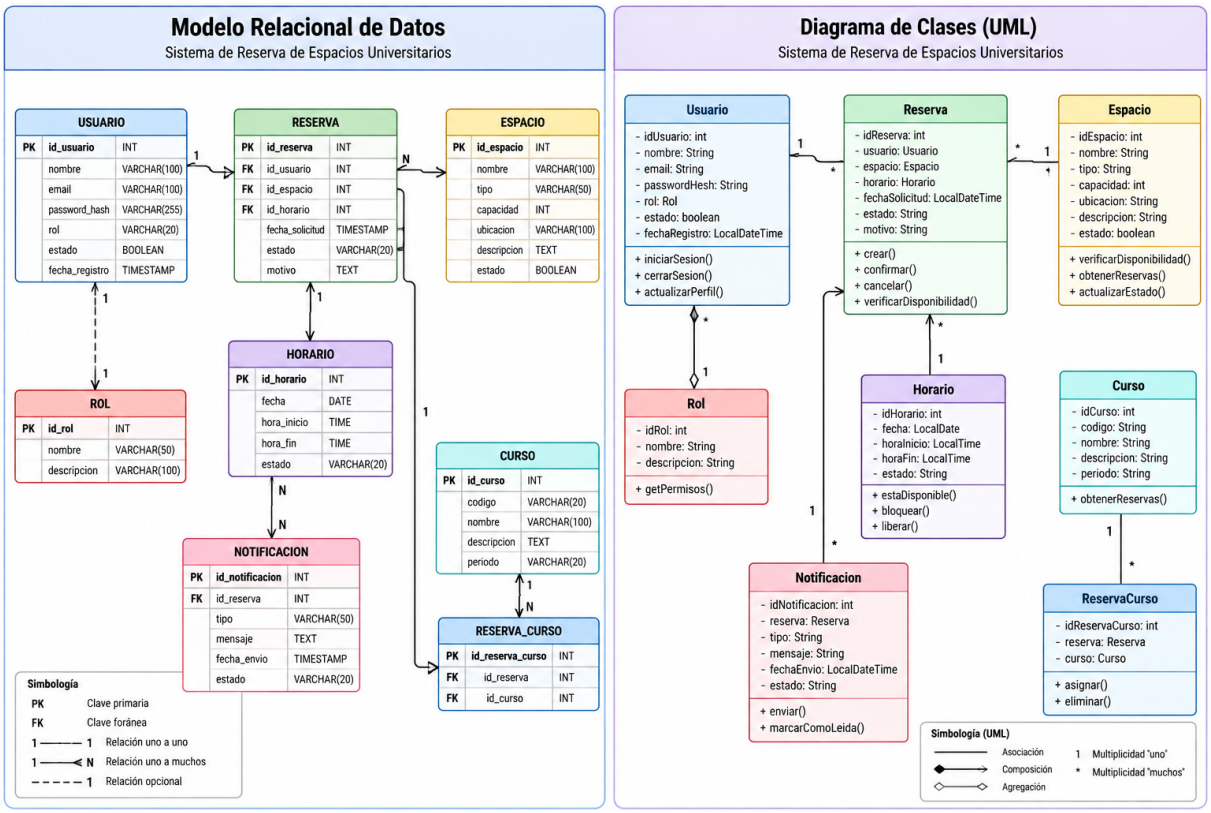


Figura 2: Modelo relacional y diagrama de clases del sistema.

6.2. Modelo Entidad–Relación

El modelo de datos identifica las entidades necesarias para mantener la persistencia del sistema.

Las relaciones permiten representar usuarios, roles, reservas, espacios, horarios, cursos y notificaciones.

La Figura 2 presenta conjuntamente el modelo relacional y su correspondencia con las clases del dominio.

7. Modelo de Descripción Detallada de los Requerimientos

Cada requerimiento funcional será especificado utilizando la siguiente estructura.

Campo	**Descripción**
Identificador	Código único RFXX
Nombre	Nombre del requerimiento
Descripción	Comportamiento esperado
Actor	Usuario o sistema participante
Entradas	Información necesaria
Precondiciones	Estado previo requerido
Proceso	Secuencia principal
Salidas	Resultado producido
Flujos alternativos	Situaciones excepcionales
Criterio de aceptación	Condición verificable de cumplimiento
Prioridad	Alta, media o baja

7.1. RF07 – Crear Reserva

Identificador	RF07
Nombre	Crear reserva
Actor	Estudiante
Entrada	Usuario, espacio, fecha y horario.
Precondición	El usuario debe encontrarse autenticado.
Proceso	El sistema recibe la solicitud, valida los datos, consulta disponibilidad y crea la reserva.
Salida	Reserva creada con identificador único.
Flujo alternativo	Si el espacio no está disponible, el sistema informa la situación y presenta alternativas.
Criterio de aceptación	Dado un espacio disponible, cuando el usuario confirma la solicitud, el sistema registra una única reserva y retorna su identificador.
Prioridad	Alta

Nota para el estudiante: esta misma estructura debe desarrollarse para RF01–RF15.

8. Diseño Conceptual del Software

El diseño conceptual transforma los requerimientos previamente identificados en modelos que describen las funcionalidades, interacciones y estructura de información del sistema.

8.1. Casos de Uso Extendidos

Cada caso de uso deberá especificar:

- identificador;
- nombre;
- objetivo;
- actores;
- precondiciones;
- postcondiciones;
- flujo principal;
- flujos alternativos;

- excepciones;
- requerimientos relacionados.

8.1.1. CU07 – Crear Reserva

Actor principal: Estudiante.

Precondición: el estudiante se encuentra autenticado.

Flujo principal:

1. El estudiante consulta los espacios.
2. Selecciona un espacio.
3. Selecciona fecha y horario.
4. El sistema verifica disponibilidad.
5. El estudiante confirma la solicitud.
6. El sistema crea la reserva.
7. El sistema almacena la reserva.
8. Se envía una confirmación.

Flujo alternativo:

Si el espacio no se encuentra disponible, el sistema informa al usuario y presenta alternativas de horario.

8.2. Diagrama de Datos Relacional

El diseño relacional se estructura mediante las tablas:

`USUARIO - ROL - RESERVA - ESPACIO - HORARIO`
`CURSO - RESERVA_CURSO - NOTIFICACION`

Las claves primarias y foráneas permiten mantener la integridad referencial entre las entidades.

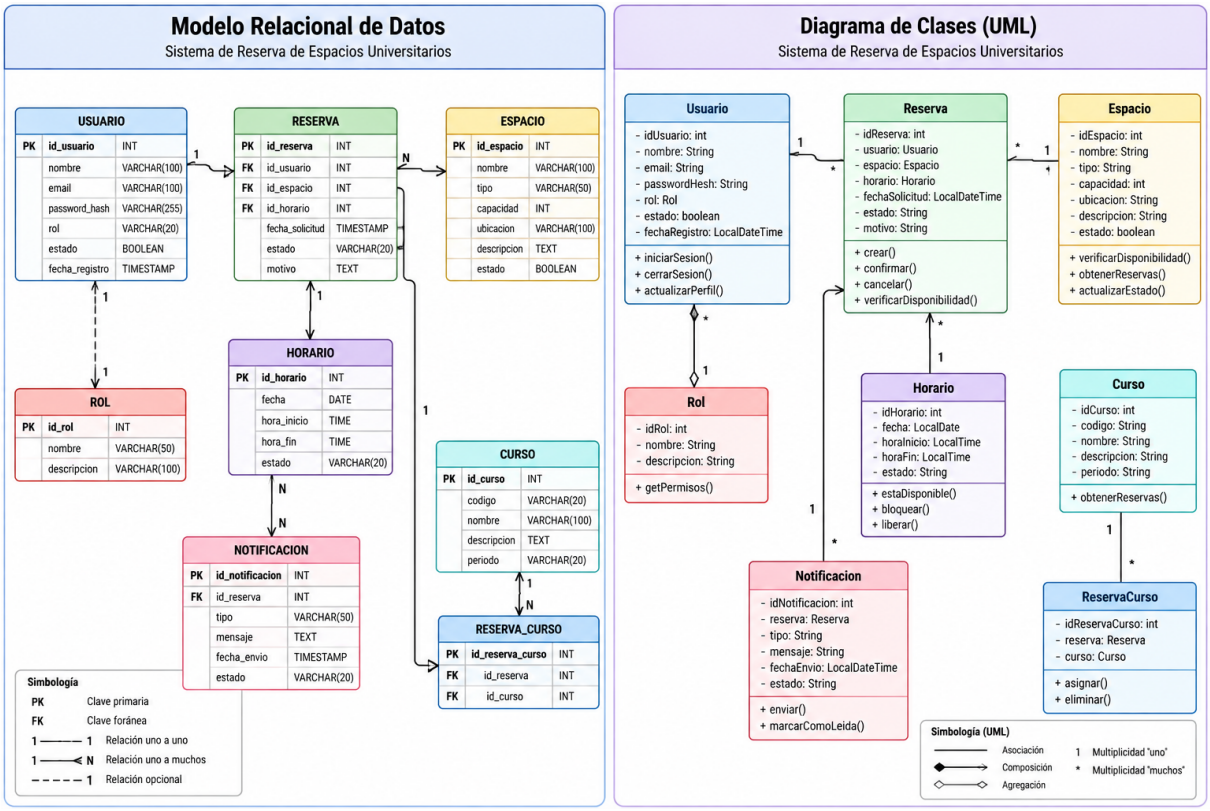


Figura 3: Diseño relacional del sistema.

9. Diseño Arquitectónico

9.1. Arquitectura del Sistema

A partir del flujo presentado en los diagramas, la arquitectura propuesta considera las siguientes capas:

Usuario

↓

Interfaz Web / Frontend

↓

API Backend / Controller

↓

Servicios de Negocio

↓

Repositorio

↓

PostgreSQL

Adicionalmente, el servicio de reservas puede comunicarse con un servicio externo de notificaciones.

9.2. Diagrama de Componentes

Los principales componentes conceptuales son:

- Frontend Web.
- API Backend.
- ReservaController.
- ReservaService.
- ReservaRepository.
- Servicio de Notificaciones.
- PostgreSQL.

La relación conceptual es:

$$Frontend \rightarrow API \rightarrow Service \rightarrow Repository \rightarrow Database$$

10. Diseño Detallado

10.1. Diagrama de Secuencia

El diagrama de secuencia representa temporalmente las interacciones necesarias para crear una reserva.

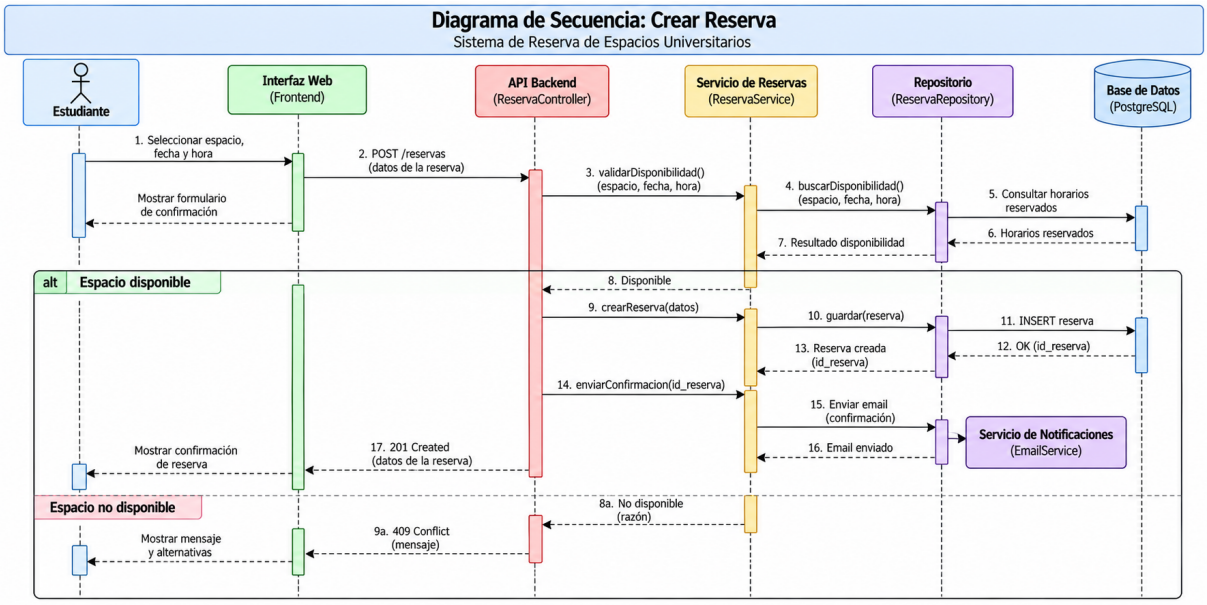


Figura 4: Diagrama de secuencia para crear una reserva.

El proceso considera dos escenarios:

1. Espacio disponible: la reserva se crea y confirma.
2. Espacio no disponible: se retorna un conflicto y se presentan alternativas.

10.2. Diagrama de Estado

Para la entidad Reserva se propone el siguiente ciclo conceptual:

$$\textbf{Solicitada} \rightarrow \textbf{Confirmada} \rightarrow \textbf{Activa} \rightarrow \textbf{Finalizada}$$

con estados alternativos:

$$\textbf{Cancelada} \qquad \textbf{Rechazada} \qquad \textbf{Expirada}$$

Nota: este diagrama deberá incorporarse como parte del avance de diseño detallado.

10.3. Diagrama de Actividad

El diagrama de actividad representa el proceso completo mediante responsabilidades distribuidas entre usuario, frontend, backend, servicio y base de datos.

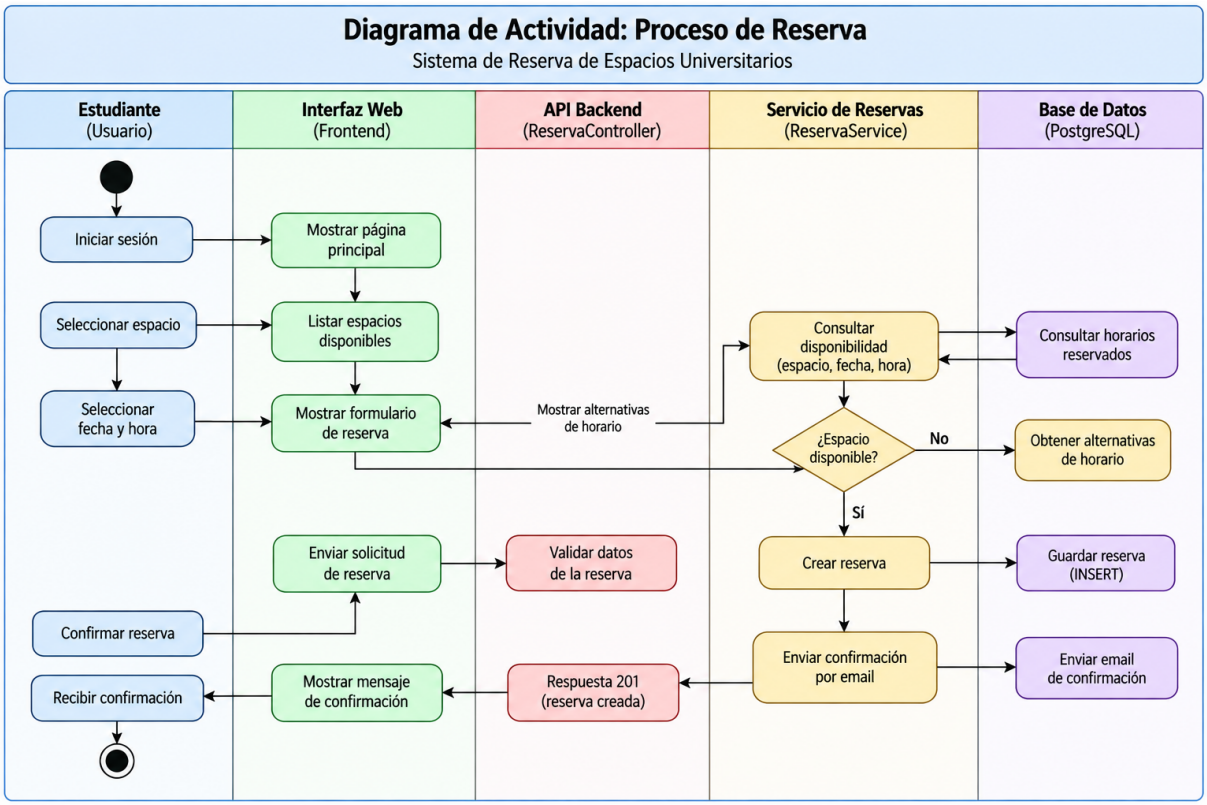


Figura 5: Diagrama de actividad del proceso de reserva.

11. Prototipo TRL-2

El prototipo TRL-2 corresponde a una representación conceptual de la solución y permitirá demostrar las principales interacciones esperadas antes de desarrollar completamente el producto.

El prototipo deberá considerar como mínimo:

- pantalla de inicio de sesión;
- página principal;
- consulta de espacios;
- visualización de disponibilidad;
- formulario de reserva;
- confirmación de reserva;
- consulta de reservas;
- panel administrativo.

En esta etapa no se requiere que todas las funcionalidades se encuentren implementadas completamente.

12. Diseño de Pruebas

12.1. Estrategias de Pruebas

La estrategia contempla:

1. Pruebas unitarias.
2. Pruebas de integración.
3. Pruebas funcionales.
4. Pruebas de sistema.
5. Pruebas de aceptación.

Cada RF deberá disponer de al menos un caso de prueba asociado.

12.1.1. Modelo de Caso de Prueba

ID	CP-RF07-01
Requerimiento	RF07 – Crear reserva
Objetivo	Verificar la creación correcta de una reserva disponible.
Precondición	Usuario autenticado y espacio disponible.
Entrada	Espacio, fecha y horario válidos.
Resultado esperado	Reserva almacenada y confirmación retornada.
Resultado obtenido	Pendiente de ejecución.
Estado	Pendiente / Aprobado / Rechazado.

12.2. Resultados de Pruebas

Los resultados deberán registrarse utilizando una matriz como:

Prueba	RF	Resultado esperado	Estado
CP01	RF01	Usuario registrado	Pendiente
CP02	RF02	Autenticación correcta	Pendiente
CP03	RF05	Disponibilidad mostrada	Pendiente
CP04	RF07	Reserva creada	Pendiente
CP05	RF09	Reserva cancelada	Pendiente
CP06	RF14	Notificación enviada	Pendiente

Matriz de Trazabilidad

La trazabilidad permite verificar que los requerimientos identificados se encuentran representados en los diferentes productos de diseño.

RF	Caso Uso	Clase	Secuencia	Prueba
RF01	Registrar usuario	Usuario	–	CP01
RF02	Iniciar sesión	Usuario	–	CP02
RF05	Consultar disponibilidad	Espacio/Horario	Crear Reserva	CP03
RF07	Crear reserva	Reserva	Crear Reserva	CP04
RF09	Cancelar reserva	Reserva	–	CP05
RF14	Enviar notificación	Notificación	Crear Reserva	CP06

13. Conclusiones

La formulación realizada permitió transformar una necesidad de gestión universitaria en una propuesta estructurada de software.

Los requerimientos funcionales establecen las capacidades principales de la solución, mientras que los requerimientos no funcionales incorporan atributos relacionados con seguridad, rendimiento, integridad, usabilidad, confiabilidad y mantenibilidad.

Los modelos de casos de uso, clases, datos, secuencia y actividad permiten observar el sistema desde perspectivas complementarias y establecer trazabilidad entre requerimientos, arquitectura, componentes y comportamiento.

Las siguientes etapas deberán avanzar desde este diseño hacia la construcción del prototipo, implementación progresiva y ejecución de los casos de prueba definidos.

14. Referencias

- Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1.*
- Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*. McGraw-Hill.
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.